

UNIVERSIDAD ABIERTA INTERAMERICANA

Facultad de Tecnología Informática

TurnoSync

Sistema de Gestión de Turnos Médicos

Carpeta de Proyecto — Ingeniería de Software

Materia:	Ingeniería de Software (IS)
Alumno:	Federico Mendez
Legajo:	B00019353-T1
Campus:	095 UAI - Castelar
Año de la carrera / Comisión / Turno:	3° "A" / Turno Noche
Año de cursada:	2026
Docente:	Julián Martín Rodríguez Escobedo
Versión del documento:	1.0

Índice

Índice.....	2
Historial de Revisión.....	4
Plan de Iteraciones — Ingeniería de Software	5
G06. Diagrama de clases parcial de todos los módulos implementados	6
Diagrama de componentes / arquitectura (capas)	6
Diagrama de clases — Módulos técnicos de BLL (Seguridad, Multi-idioma, Bitácora, Encriptado).....	7
Responsabilidades de las clases principales (T04).....	8
Patrón Observer — detalle de implementación (Multi-idioma)	8
Diagrama de clases — Gestiones de negocio (Paciente, Profesional, Especialidad, Turno)	9
Diagrama de secuencia — T02. Login (ciclo completo: arranque / login / logout).....	9
G07. Modelo de datos parcial de todos los módulos implementados.....	11
Notas sobre cardinalidad y modalidad	11
Documentos de Aspectos Técnicos	13
T01. Arquitectura Base	13
a. Objetivo	13
b. Descripción detallada de cómo funciona.....	13
c. Diagrama de componentes	13
Mapa de navegación de menús	13
Pruebas unitarias de arquitectura	14
T02. Gestión de Login/Logout y gestión de usuarios	15
a. Objetivo	15
b. Descripción detallada de cómo funciona.....	15
c. Diagrama de clases.....	15
d. Diagrama de secuencia.....	15
T03. Gestión de Encriptado	16
a. Objetivo	16
b. Descripción detallada de cómo funciona.....	16
T04. Gestión de Perfiles de Usuario	17
a. Objetivo	17
b. Descripción detallada de cómo funciona.....	17
Caso de uso técnico — Asignar Permisos a un Grupo (Rol).....	17
T05. Gestión de Múltiples Idiomas.....	19
a. Objetivo	19

b. Descripción detallada de cómo funciona.....	19
T06. Gestión de Bitácora y Control de Cambios.....	20
T06a. Gestión de Bitácora.....	20
T06b. Control de Cambios	20
T07 / T08. Gestión de Dígitos Verificadores	22
a. Objetivo	22
b. Descripción detallada de cómo funciona.....	22
Verificación de integridad y restauración.....	23
Evaluación y Retrospectiva	24

Historial de Revisión

Fecha	Versión	Autor	Descripción
22/04/2026	0.1	Federico Mendez	1° Entrega — arquitectura base (T01), Login/Logout (T02), Bitácora inicial (T06a), Encriptado (T03)
10/06/2026	0.2	Federico Mendez	2° Entrega — Perfiles de Usuario (T04), Multi-idioma (T05), Dígitos Verificadores (T07), Control de Cambios inicial (T06b)
08/07/2026	1.0	Federico Mendez	3° Entrega (final) — Control de Cambios completo con serialización, corrección de fórmula DVH, pruebas unitarias, logging de excepciones, integración G06/G07

Plan de Iteraciones — Ingeniería de Software

El desarrollo se organizó en 3 entregas, siguiendo el cronograma de la cátedra (Campus 095 UAI - Castelar, 3° "A", Turno Noche). Esta carpeta corresponde solo a Ingeniería de Software: los ítems de Trabajo de Diploma (G01-G05, N01-N02, A01-A03, D01-D03) se cursan el cuatrimestre que viene y no están acá. Lo que sigue es T01 a T08 y la integración de diagramas de clases y modelo de datos (G06/G07).

Iteración	Ítems desarrollados	Fecha
1° Entrega	T01. Arquitectura Base · T02. Login/Logout (Singleton) · T06a. Bitácora · T03. Encriptado	22/04/2026
2° Entrega	T06. Bitácora y Control de Cambios (refinar) · T07. Dígitos Verificadores · T04. Perfiles de Usuario (Composite) · T06b. Control de Cambios · T05. Multi-idioma (Observer)	10/06/2026
3° Entrega (final)	T04 (refinar) · T08. Dígitos Verificadores (refinar) · T05 (refinar) · G06. Diagrama de clases parcial · G07. Modelo de datos parcial · Evaluación y retrospectiva	08/07/2026

G06. Diagrama de clases parcial de todos los módulos implementados

Acá junto los diagramas de clases de cada módulo técnico que llevo desarrollado, separando lo técnico del negocio y con un diagrama por capa, como pide la consigna.

Diagrama de componentes / arquitectura (capas)

Diagrama de Componentes — Arquitectura TurnoSync (4 capas)

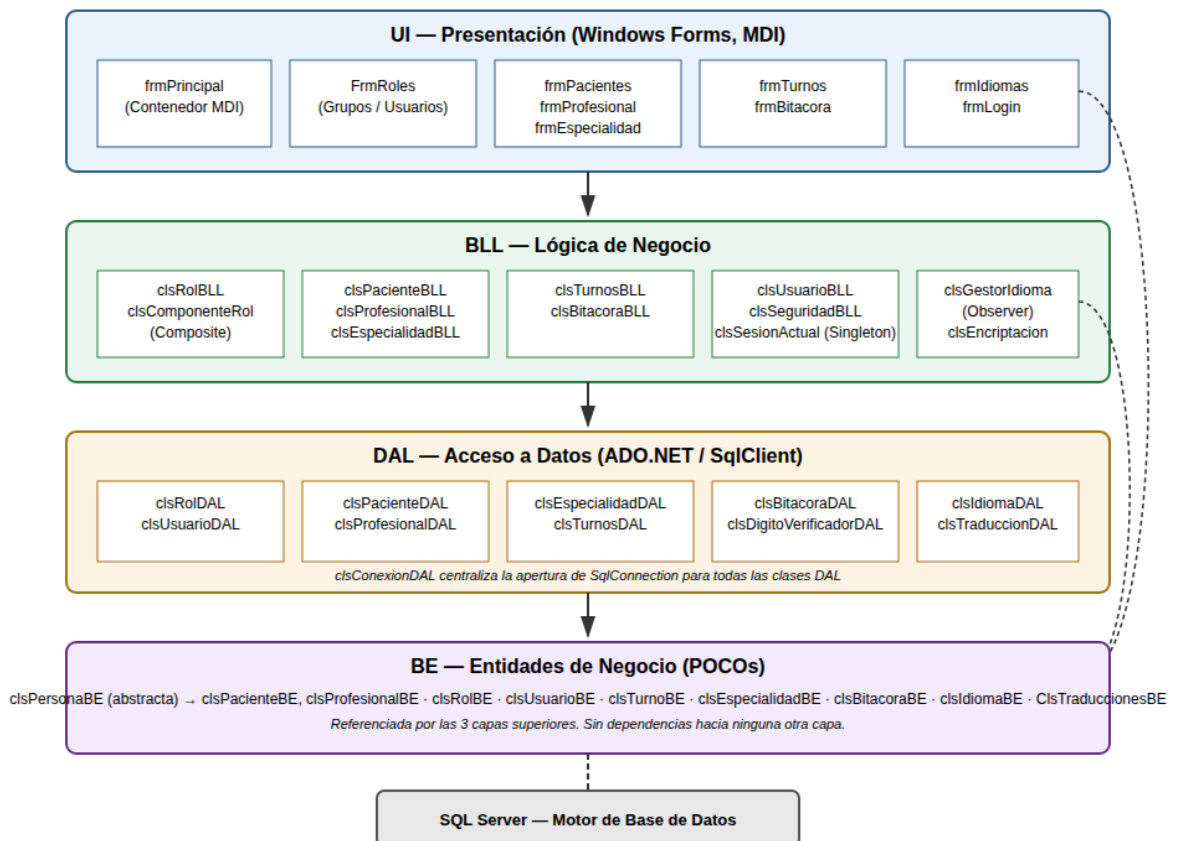
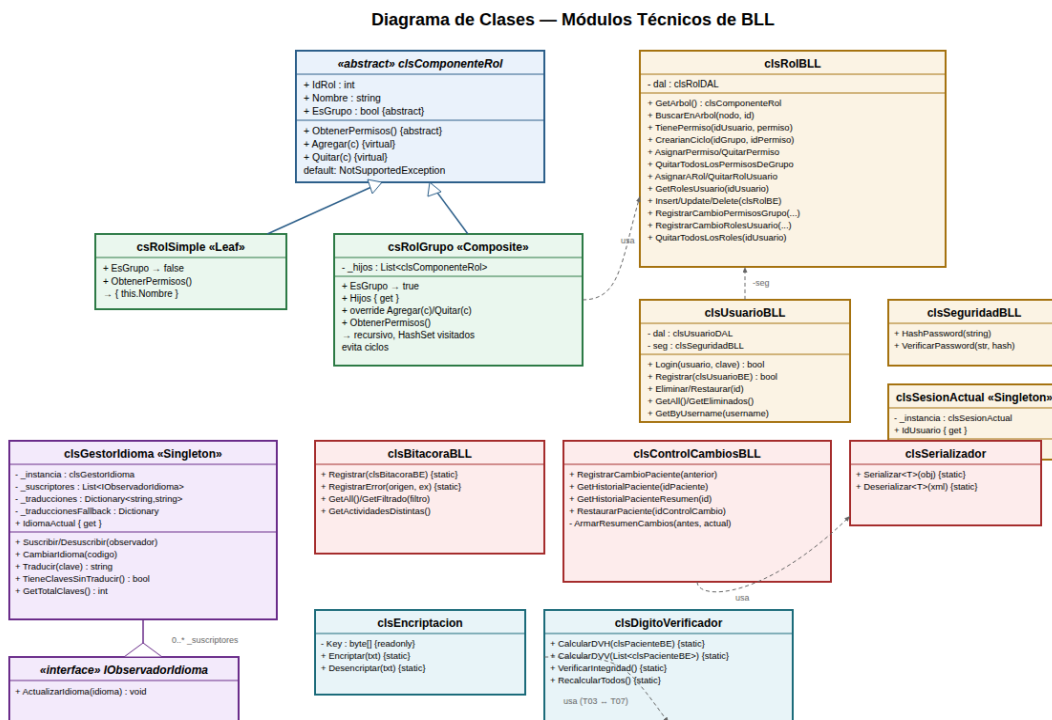


Diagrama de componentes de la arquitectura en 4 capas.

Diagrama de clases — Módulos técnicos de BLL (Seguridad, Multi-idioma, Bitácora, Encriptado)

Se agrupó en un mismo diagrama toda la capa BLL correspondiente a los aspectos técnicos transversales del sistema, separada de las clases de negocio (que se muestran en un diagrama aparte, más abajo). Incluye: el patrón Composite completo (clsComponenteRol, csRolSimple, csRolGrupo, clsRolBLL) para la gestión de perfiles; Login y Singleton (clsUsuarioBLL, clsSeguridadBLL, clsSesionActual); el patrón Observer para multi-idioma (clsGestorIdioma, IObservadorIdioma); Bitácora y Control de Cambios (clsBitacoraBLL, clsControlCambiosBLL, clsSerializador); y Encriptado y Dígitos Verificadores (clsEncriptacion, clsDigitoVerificador).



Composite (clsComponenteRol/csRolSimple/csRolGrupo) - Singleton (clsSesionActual, clsGestorIdioma) - Observer (clsGestorIdioma/IObservadorIdioma)

Diagrama de clases de los módulos técnicos de BLL (Composite, Singleton, Observer, Bitácora/Control de Cambios, Encriptado/DVH).

Responsabilidades de las clases principales (T04)

Clase	Responsabilidad (hacer / conocer)
clsComponenteRol (abstracta)	Conocer: Id y nombre de un permiso. Hacer: definir el contrato común (ObtenerPermisos) que toda hoja o grupo debe cumplir.
csRolSimple (Leaf)	Conocer: su propio nombre. Hacer: devolverse a sí mismo como único permiso otorgado (no tiene hijos).
csRolGrupo (Composite)	Conocer: la lista de componentes hijos que agrupa. Hacer: agregar/quitar hijos, y calcular recursivamente el conjunto total de permisos que otorga (incluyendo los de sus subgrupos), evitando ciclos mediante un conjunto de nodos visitados.
clsRolBLL	Hacer: construir el árbol completo desde la base de datos, buscar un nodo por Id, determinar si un usuario tiene un permiso dado, detectar si una asignación crearía un ciclo antes de guardarla.
clsRolDAL	Hacer: persistir y recuperar las relaciones Rol / RolPermiso / UsuarioRol contra SQL Server.
clsUsuarioBLL / clsUsuarioDAL	Hacer: autenticar usuarios (Login, delegando el hash a clsSeguridadBLL), gestionar altas/bajas lógicas de cuentas.

Patrón Observer — detalle de implementación (Multi-idioma)

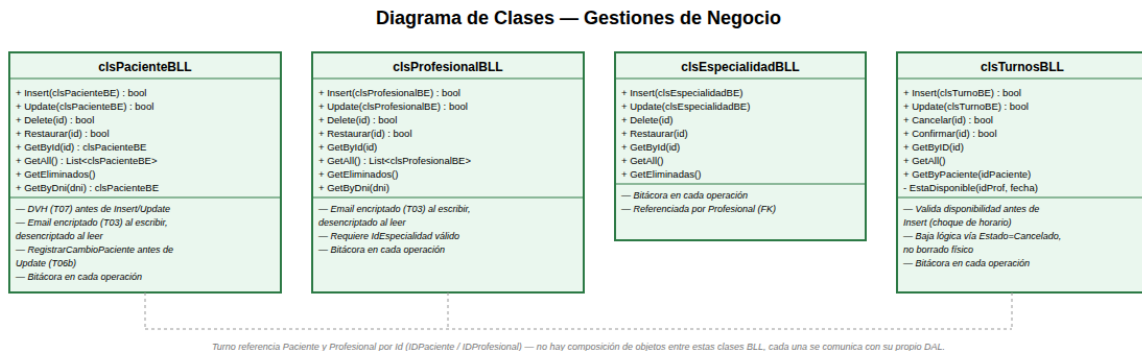
Las clases clsGestorIdioma e IObservadorIdioma ya están en el diagrama de clases anterior (BLL Técnico). clsGestorIdioma es el sujeto del patrón Observer: mantiene una lista de formularios suscriptos (los que implementan IObservadorIdioma) y, cuando cambia el idioma activo, les avisa a todos para que refresquen sus textos. Ninguno de los dos lados conoce al otro directamente.

```
public interface IObservadorIdioma
{
    void ActualizarIdioma(string idioma);
}

public class clsGestorIdioma
{
    private List<IObservadorIdioma> observadores = new();
    public void Suscribirse(IObservadorIdioma o) => observadores.Add(o);
    public void Desuscribirse(IObservadorIdioma o) => observadores.Remove(o);
    public void CambiarIdioma(string idioma)
    {
        IdiomaActual = idioma;
        foreach (var o in observadores) o.ActualizarIdioma(idioma);
    }
}
```


Diagrama de clases — Gestiones de negocio (Paciente, Profesional, Especialidad, Turno)

Se agrupan acá las clases de negocio propiamente dichas, separadas de los aspectos técnicos de los diagramas anteriores. Cada una sigue el mismo patrón: validaciones en los métodos de escritura, registro en Bitácora, y en Paciente/Profesional, el manejo del email encriptado (T03).



Las 4 clases siguen el mismo patrón: validar → operar vía DAL, propio → registrar en Bitácora. No comparten herencia entre sí (a diferencia de Paciente/Profesional a nivel BE, que sí heredan de clsPersonaBE).

Diagrama de clases de las gestiones de negocio (Paciente, Profesional, Especialidad, Turno).

Diagrama de secuencia — T02. Login (ciclo completo: arranque / login / logout)

La consigna de T02 pide diferenciar explícitamente los procesos que corren en el arranque del sistema, el log-in, y el apagado (log-out) — incluyendo permisos, auditoría y control de integridad. Por eso el diagrama de secuencia no se limita al intercambio de credenciales: muestra las 4 fases completas del ciclo de vida de una sesión.

- Arranque: VerificarIntegridad() (T07/T08) corre antes de mostrar cualquier ventana; si falla, la aplicación no llega a mostrar el login.
- Log-in: verificación de usuario y contraseña (delegada a clsSeguridadBLL), registro en Bitácora, e inicio de sesión en el Singleton clsSesionActual.
- Post-login: frmPrincipal consulta clsRolBLL.TienePermiso() por cada opción de menú, para habilitar solo lo que el usuario tiene permitido (integración con T04).
- Log-out: al cerrar frmPrincipal, se finaliza el proceso completo — no hay cambio de usuario sin reiniciar la aplicación, ya que el Singleton vive durante todo el proceso.

Diagrama de Secuencia — Ciclo Completo: Arranque / Login / Logout (T02)

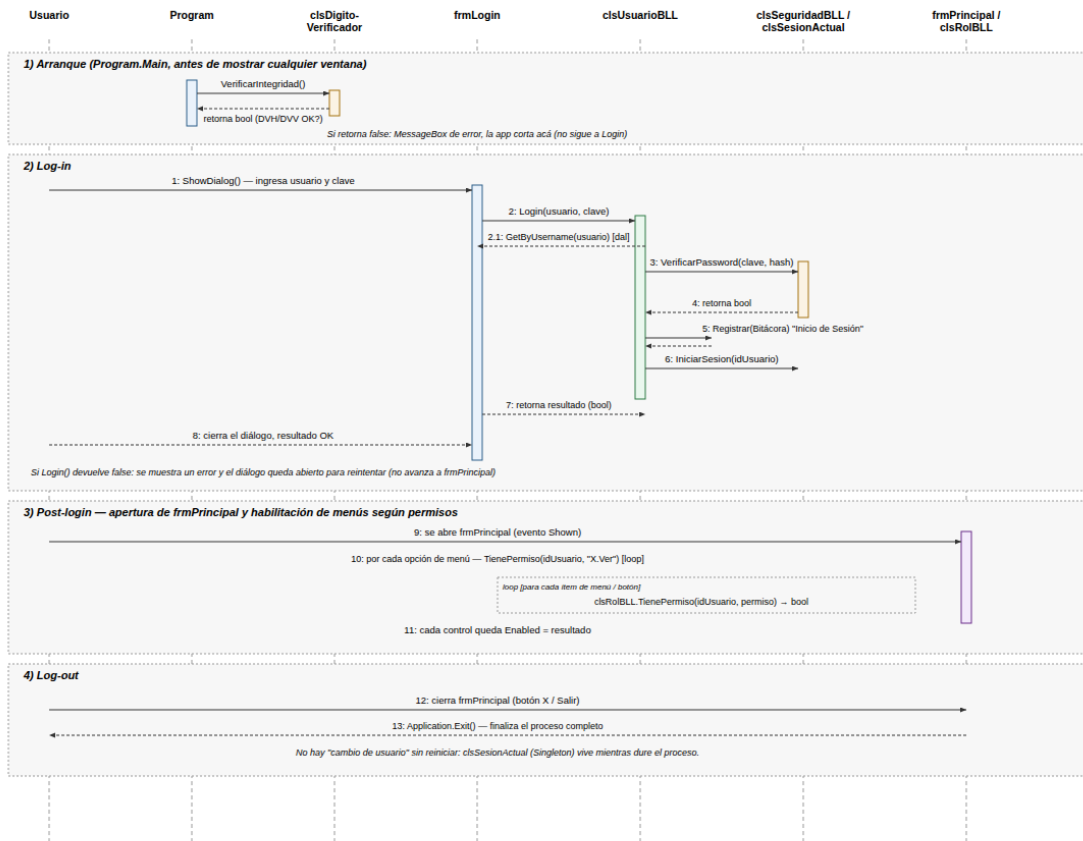


Diagrama de secuencia del ciclo completo: arranque, login, post-login (permisos) y logout.

G07. Modelo de datos parcial de todos los módulos implementados

El DER completo, con lo técnico (Seguridad/Permisos, Bitácora, Dígitos Verificadores, Multi-idioma) separado visualmente del negocio (Paciente, Profesional, Especialidad, Turno).

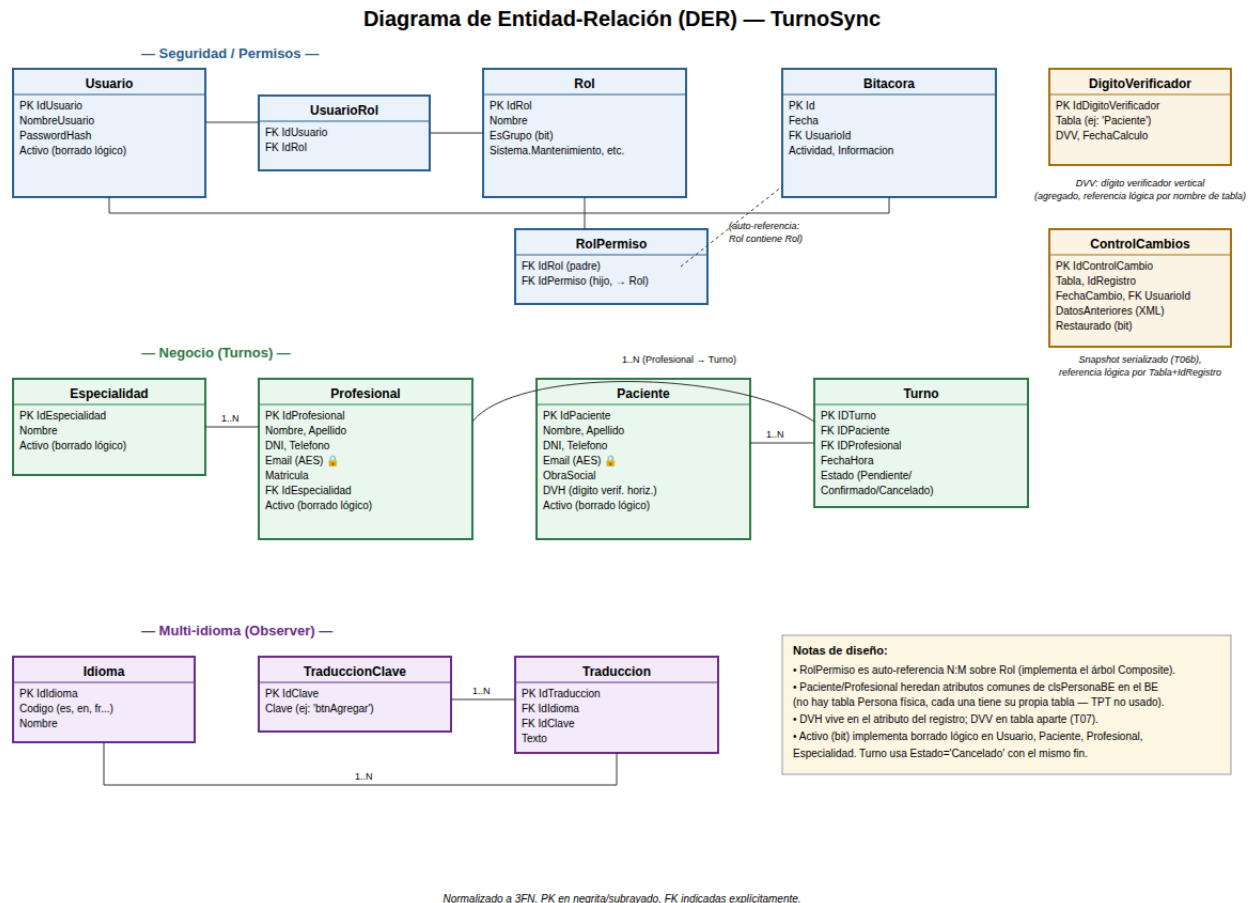


Diagrama de Entidad-Relación (DER), con todos los módulos desarrollados hasta esta entrega.

Notas sobre cardinalidad y modalidad

- Usuario ↔ Rol: relación N:M resuelta por UsuarioRol (un usuario puede tener varios roles; un rol puede estar asignado a varios usuarios).
- Rol ↔ Rol: auto-relación N:M resuelta por RolPermiso, que es la que materializa el árbol Composite en la base de datos (un grupo puede contener 1..N permisos; un permiso puede pertenecer a 0..N grupos).
- Profesional → Especialidad: relación N:1 (un profesional tiene exactamente una especialidad; una especialidad puede tener 0..N profesionales).
- Turno → Paciente / Turno → Profesional: ambas N:1 (un turno referencia exactamente un paciente y un profesional; cada uno puede tener 0..N turnos).
- Bitacora → Usuario: N:1 (cada entrada de bitácora pertenece a un único usuario; un usuario puede generar 0..N entradas).

- Traduccion → Idioma / Traduccion → TraduccionClave: ambas N:1 (cada traducción pertenece a un idioma y a una clave; cada idioma/clave tiene 0..N traducciones asociadas).

Documentos de Aspectos Técnicos

Acá documento cada gestión técnica transversal del sistema: qué resuelve, cómo la implementé, y las decisiones de diseño que tomé en el camino. Sigo el orden del Plan de Entregas de la cátedra (T01 a T08).

T01. Arquitectura Base

a. Objetivo

Que la interfaz, la lógica de negocio y el acceso a datos queden separados en capas independientes, para poder tocar una sin que eso obligue a tocar las demás.

b. Descripción detallada de cómo funciona

La solución tiene 4 proyectos (.csproj) separados:

- UI: los formularios Windows Forms. frmPrincipal es el contenedor MDI del que cuelgan todas las pantallas.
- BLL: la lógica de negocio y las validaciones. Es la única capa que UI puede referenciar directamente.
- DAL: abre la conexión, ejecuta los comandos SQL parametrizados y mapea el resultado a objetos BE. No conoce nada de BLL — de hecho DAL.csproj solo referencia a BE.csproj, así que ni siquiera podría acoplarse aunque quisiera.
- BE: las entidades, sin comportamiento, solo propiedades. Las usan las tres capas de arriba.

Para leer datos, cada método del DAL abre una SqlConnection (siempre a través de clsConexionDAL, que centraliza el string de conexión), ejecuta el comando y arma los objetos con un Mapear(dr) privado. No usé ningún ORM — todo el mapeo es manual.

Para escribir, en cambio, cada método abre una transacción explícita y hace Commit o Rollback según el resultado:

```
using (SqlConnection con = clsConexionDAL.GetConnection())
{
    con.Open();
    SqlTransaction tran = con.BeginTransaction();
    try
    {
        SqlCommand cmd = new SqlCommand("UPDATE Paciente SET Activo = 0 WHERE IdPaciente = @Id", con, tran);
        cmd.Parameters.AddWithValue("@Id", idPaciente);
        cmd.ExecuteNonQuery();
        tran.Commit();
        return true;
    }
    catch { tran.Rollback(); return false; }
}
```

c. Diagrama de componentes

Está en la sección G06, junto con el resto de los diagramas de clases.

Mapa de navegación de menús

Código	GUI	Acceso desde
--------	-----	--------------

M01	frmLogin	Inicio de la aplicación
M02	frmPrincipal (MDI)	Login exitoso
M02.01	frmPacientes	Menú Gestiones → Pacientes
M02.02	frmProfesional	Menú Gestiones → Profesionales
M02.03	frmEspecialidad	Menú Gestiones → Especialidades
M02.04	frmTurnos	Menú Gestiones → Turnos
M02.05	FrmRoles	Menú Gestiones → Roles (pestañas Grupos / Usuarios)
M02.06	frmIdiomas	Menú Gestiones → Idiomas (pestañas Languages / Translations)
M02.07	frmBitacora	Menú Informes → Bitácora

Pruebas unitarias de arquitectura

Armé un proyecto de pruebas aparte (PruebasUnitarias, con MSTest) con 13 tests, en dos grupos:

- 5 pruebas de arquitectura, que revisan por reflection los ensamblados compilados: que DAL no dependa de BLL ni de UI, que BE no dependa de nada, y que BLL sí dependa de DAL pero no de UI. Si algún día por error alguien mete una referencia cruzada, estas pruebas la detectan.
- 8 pruebas de lógica, que corren en memoria sin tocar la base: que el Composite arma bien el árbol de permisos y no se cuelga si alguien crea un ciclo entre grupos; que el DVH no cambia según el Id del registro; que el DNI manipulado se detecta; que AES es reversible y da un resultado distinto cada vez que se llama (por el IV aleatorio); y que clsSesionActual efectivamente devuelve siempre la misma instancia.

Las 13 corren en un par de segundos y no necesitan conexión a SQL Server, así que las uso como chequeo rápido antes de cada entrega.

T02. Gestión de Login/Logout y gestión de usuarios

a. Objetivo

Confirmar quién es el usuario antes de dejarlo entrar, y con qué permisos cuenta según los roles que tenga asignados.

b. Descripción detallada de cómo funciona

Cuando arranca la aplicación (Program.cs), antes de mostrar cualquier ventana corre la verificación de integridad de la base (T07/T08). Si todo está bien, recién ahí se abre frmLogin de forma modal. El usuario pone su nombre y clave; clsUsuarioBLL.Login() busca la cuenta (dejando afuera las dadas de baja) y compara el hash SHA-256 de la clave ingresada contra el guardado (T03). Cada intento, sea exitoso o no, queda registrado en la Bitácora.

Si el login funciona, el usuario autenticado se guarda en clsSesionActual — el Singleton — para que cualquier pantalla pueda consultar quién está logueado sin que se lo tengan que pasar por parámetro. Recién ahí se abre frmPrincipal, que en su evento Shown consulta los permisos del usuario y habilita o deshabilita cada menú y cada botón según el árbol de T04.

El logout es simplemente cerrar frmPrincipal: termina el proceso y se acabó la sesión. No hay opción de cambiar de usuario sin reiniciar la app.

c. Diagrama de clases

```
public class clsSesionActual
{
    private static clsSesionActual instancia;
    public int IdUsuario { get; private set; }

    private clsSesionActual() { }

    public static clsSesionActual GetInstancia()
    {
        if (instancia == null) instancia = new clsSesionActual();
        return instancia;
    }

    public void IniciarSesion(int idUsuario) => IdUsuario = idUsuario;
}
```

El constructor es privado y solo se puede llegar a la instancia por GetInstancia() — es el Singleton clásico, y evita andar pasando el usuario logueado de formulario en formulario.

d. Diagrama de secuencia

Está en la sección G06.

T03. Gestión de Encriptado

a. Objetivo

Proteger los datos sensibles de la base: las contraseñas de los usuarios y los datos de contacto de pacientes y profesionales.

b. Descripción detallada de cómo funciona

Usé dos mecanismos distintos, porque no son el mismo problema:

- Para las contraseñas, hash de una vía (SHA-256). Nunca hace falta recuperar la contraseña original, solo comparar. `clsSeguridadBLL.HashPassword()` calcula el hash al crear el usuario, y `VerificarPassword()` recalcula el de la clave ingresada y la compara contra la guardada.
- Para el email de Pacientes y Profesionales, cifrado simétrico reversible (AES-256) — porque a diferencia de la contraseña, el email sí hay que poder mostrarlo en pantalla.

`clsEncriptacion` usa AES en modo CBC, con clave fija de 256 bits pero IV aleatorio en cada llamada (no fijo), porque el email no necesita buscarse por igualdad exacta en SQL — a diferencia del DNI, que por eso quedó sin encriptar. El IV va pegado adelante del texto cifrado, codificado todo junto en Base64, así al desencriptar se puede recuperar sin guardarlo aparte.

```
public static string Encriptar(string textoPlano)
{
    using (Aes aes = Aes.Create())
    {
        aes.Key = Key;           // 256 bits, fija
        aes.GenerateIV();        // aleatorio en cada llamada
        aes.Mode = CipherMode.CBC;
        aes.Padding = PaddingMode.PKCS7;
        using var encryptor = aes.CreateEncryptor();
        using var ms = new MemoryStream();
        ms.Write(aes.IV, 0, aes.IV.Length); // IV al principio del blob
        using (var cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
        using (var sw = new StreamWriter(cs)) sw.Write(textoPlano);
        return Convert.ToBase64String(ms.ToArray());
    }
}
```

El cifrado y descifrado pasan enteramente por el BLL (`clsPacienteBLL`, `clsProfesionalBLL`), justo antes de mandarle el dato al DAL para guardar, y justo después de recibirlo del DAL para leer. El DAL nunca se entera de que el dato viaja cifrado. Y el dígito verificador (T07/T08) siempre se calcula sobre el valor real, nunca sobre el texto cifrado.

T04. Gestión de Perfiles de Usuario

a. Objetivo

Poder asignar permisos a los usuarios de forma flexible, combinando permisos sueltos y grupos de permisos, sin tener que tocar código cada vez que aparece una combinación nueva.

b. Descripción detallada de cómo funciona

Para esto usé el patrón Composite (diagrama en G06). Cada nodo del árbol de permisos es un `clsComponenteRol`, que puede ser una hoja (`csRolSimple` — un permiso atómico como `Pacientes.Agregar`) o un grupo (`csRolGrupo` — como `GestionPacientes` o `Administrador`, que a su vez agrupan otros nodos).

La relación padre-hijo vive en la tabla `RolPermiso` (una auto-relación N:M sobre `Rol`). `clsRolBLL.GetArbol()` arma el árbol completo en memoria en dos pasadas: primero crea un nodo por cada fila de `Rol`, y después recorre `RolPermiso` conectando cada hijo con su grupo padre. La raíz siempre es el nodo "Sistema".

Para saber si un usuario tiene un permiso, `TienePermiso(idUsuario, permiso)` recorre los roles que tiene asignados y llama a `ObtenerPermisos()` sobre cada uno — un método recursivo que, si es un grupo, junta los permisos de todos sus hijos. Lleva un `HashSet` de nodos ya visitados para no colgarse si en algún momento se arma un ciclo por error entre grupos (algo que efectivamente me pasó probando y quedó cubierto por un test).

La pantalla de asignación usa un `TreeView` con checkboxes que se arma recursivamente desde el árbol. Antes de guardar, `CrearCiclo(idGrupo, idPermisoNuevo)` chequea si el grupo destino ya es alcanzable desde el permiso que se quiere agregar — si es así, rechaza la operación (por ejemplo, el grupo A no puede contener a B si B ya contiene a A). Al guardar, borra todas las asignaciones previas del grupo y reinserta solo las que quedaron tildadas, para que lo que se ve en pantalla y lo que hay en la base nunca queden desincronizados.

Así se ve la nomenclatura de permisos que terminé usando:

Código / Nombre	Tipo	Contenido
Pacientes.Ver / Agregar / Modificar / Eliminar	Atómico	—
GestionPacientes	Compuesto	Pacientes.Ver, Pacientes.Agregar, Pacientes.Modificar, Pacientes.Eliminar
Administrador	Compuesto	GestionPacientes, GestionProfesionales, GestionEspecialidades, GestionTurnos, GestionUsuarios, GestionRoles, GestionBitacora

Caso de uso técnico — Asignar Permisos a un Grupo (Rol)

Campo	Detalle
Actor principal	Administrador

Precondición	El administrador tiene el permiso Roles.Asignar
Flujo principal	1) Selecciona un grupo (rol) de la lista. 2) Marca/desmarca permisos en el árbol interactivo. 3) Confirma. 4) El sistema valida que la asignación no genere un ciclo en la jerarquía, borra las asignaciones previas del grupo, y guarda las nuevas.
Flujo alternativo	3a) Si la asignación generaría un ciclo, el sistema rechaza la operación con un mensaje explicativo.
Postcondición	El grupo queda con el nuevo conjunto de permisos directos; el cambio queda auditado en Bitácora con el detalle de antes/después.

T05. Gestión de Múltiples Idiomas

a. Objetivo

Que la interfaz cambie de idioma en el momento, sin reiniciar la app, y sin que cada formulario tenga que saber cómo funciona la traducción.

b. Descripción detallada de cómo funciona

Usé el patrón Observer (diagrama en G06). Descarté a propósito los .resx por idioma, porque agregar un idioma nuevo hubiera significado recompilar la aplicación.

El modelo separa la clave de traducción (TraduccionClave, por ejemplo btnAgregar) del texto en cada idioma (Traduccion, con FK a Idioma y a TraduccionClave — ver G07). Cada formulario que se traduce implementa IObservadorIdioma y se suscribe a clsGestorIdioma al mostrarse, desuscribiéndose al cerrar para no dejar referencias colgadas.

frmIdiomas tiene un escáner por reflection que recorre todos los Form del ensamblado UI (con Activator.CreateInstance), instancia cada uno y extrae los Button, Label, GroupBox, TabPage e ítems de menú, generando las claves que faltan en TraduccionClave y una fila vacía en Traduccion por cada idioma existente. Así, si agrego un idioma nuevo o aparece una clave nueva, quedan sincronizados solos, sin que tenga que ir formulario por formulario a mano.

Como completar la traducción de cada clave en cada idioma es un trabajo que se hace de a poco, Traducir(clave) resuelve en 3 pasos: si el idioma activo tiene esa clave con texto, la devuelve; si está vacía, cae al diccionario de inglés; y si tampoco está ahí, devuelve la clave sin traducir como último recurso. También agregué un aviso: si al cambiar de idioma quedan claves sin completar, se lo informa al usuario para que sepa que esos textos van a aparecer en inglés y no piense que es un error del sistema.

T06. Gestión de Bitácora y Control de Cambios

T06a. Gestión de Bitácora

Objetivo: dejar registrada toda operación relevante que haga un usuario, para poder auditar la actividad del sistema.

Cada método de escritura de las capas de negocio (Insert/Update/Delete/Restaurar en clsPacienteBLL, clsProfesionalBLL, clsEspecialidadBLL, clsTurnosBLL, clsUsuarioBLL, clsRolBLL) arma, después de llamar al DAL, un clsBitacoraBE con la fecha y hora (las pone clsBitacoraBLL.Registrar), el usuario de clsSesionActual, un nombre de actividad en español bajo la misma convención en todos lados (Alta/Baja/Modificación de <Entidad>), y un campo de información libre que en las modificaciones detalla el antes y el después del cambio. Con esos 5 campos cubro lo mínimo que pide la consigna.

frmBitacora permite buscar combinando fecha, usuario y actividad al mismo tiempo, con una consulta donde cada filtro se ignora si no se especificó:

```
SELECT * FROM Bitacora
WHERE (@FechaDesde IS NULL OR Fecha >= @FechaDesde)
  AND (@FechaHasta IS NULL OR Fecha <= @FechaHasta)
  AND (@UsuarioId IS NULL OR UsuarioId = @UsuarioId)
  AND (@Actividad IS NULL OR Actividad = @Actividad)
ORDER BY Fecha DESC
```

También agregué manejo de excepciones en todas las capas BLL: si algo falla de forma inesperada (por ejemplo, la conexión a la base), clsBitacoraBLL.RegistrarError(origen, ex) deja constancia en la Bitácora con la actividad "Error del Sistema" y el mensaje de la excepción, antes de devolver un resultado controlado. Así ninguna excepción llega cruda a la interfaz, y de paso queda un rastro de cuándo y por qué falló algo.

T06b. Control de Cambios

Objetivo: poder responder "quién, cuándo y qué" se modificó, con la posibilidad de volver a un estado anterior.

Lo resolví en dos frentes distintos, según el tipo de cambio.

Para las bajas, con borrado lógico: Usuario, Paciente, Profesional y Especialidad tienen un campo Activo (bit). Eliminar no borra la fila, la marca Activo = 0, y las consultas normales la filtran (GetAll trae solo WHERE Activo = 1), sin perder ningún dato ni ninguna relación. Hay una pantalla de "Ver eliminados" que lista lo dado de baja, con un botón Restaurar que revierte la baja y recupera el registro tal cual estaba.

Para las ediciones, con serialización del estado anterior: justo antes de aplicar un Update sobre un Paciente, clsControlCambiosBLL.RegistrarCambioPaciente() toma el objeto completo tal como está en ese momento y lo serializa a XML, guardando el snapshot en una tabla nueva, ControlCambios (Tabla, IdRegistro, FechaCambio, UsuarioId, DatosAnteriores, Restaurado). A diferencia del texto de la Bitácora, acá guardo el objeto entero y tipado, no un resumen — eso es lo que permite reconstruirlo después.

```
public static class clsSerializador
{
```

```
public static string Serializar<T>(T objeto)
{
    var serializer = new XmlSerializer(typeof(T));
    using (var sw = new StringWriter())
    {
        serializer.Serialize(sw, objeto);
        return sw.ToString();
    }
}

public static T Deserializar<T>(string xml)
{
    var serializer = new XmlSerializer(typeof(T));
    using (var sr = new StringReader(xml))
        return (T)serializer.Deserialize(sr);
}
}
```

La pantalla frmHistorialCambios (se abre desde "Ver Historial" en la ficha de un paciente) lista los cambios de ese paciente, comparando cada snapshot viejo contra el estado actual y armando un resumen legible campo por campo (por ejemplo, Teléfono: "1122334455" → "1158471020"), sin que el usuario tenga que leer el XML. Desde ahí, Restaurar deserializa el snapshot elegido, reconstruye el objeto y lo guarda con un Update normal — el registro vuelve exactamente al estado de ese momento. Cada restauración marca el registro como Restaurado y queda auditada en Bitácora.

Esta misma serialización XML es la que uso para cubrir el requisito de Serialización: en vez de armar un archivo suelto sin ningún uso real, la apliqué a resolver el problema concreto de reconstruir el historial de una entidad.

Por ahora esto está implementado sobre Paciente, como entidad de referencia (mismo criterio que usé en Dígitos Verificadores: concentrar el control en la entidad más sensible). El mismo esquema — tabla ControlCambios genérica por Tabla + IdRegistro, más un método de registro y uno de restauración por entidad — se puede replicar directo a Profesional y Especialidad.

T07 / T08. Gestión de Dígitos Verificadores

Los documento juntos porque son el mismo módulo en dos momentos: T07 lo implementé en la 2° Entrega (cálculo y verificación básica), y en esta 3° Entrega (T08) corregí un error de diseño en la fórmula del DVH y lo integré con el encriptado de T03.

a. Objetivo

Detectar si alguien tocó los datos de la base por fuera del sistema — por ejemplo con un UPDATE directo en SQL — ya sea agregando o quitando registros, o intercambiando valores de posición.

b. Descripción detallada de cómo funciona

Lo apliqué sobre Paciente, que es donde están los datos más sensibles.

El DVH (dígito verificador horizontal) vive en un atributo de cada fila de Paciente. Se calcula recorriendo los atributos en un orden fijo (Nombre, Apellido, DNI, Teléfono, Email, Fecha de Nacimiento, Obra Social): para cada carácter se suma su código ASCII multiplicado por la posición del carácter y por la posición del atributo, y el total se reduce con módulo 97. Al entrar tanto la posición del carácter como la del atributo en la cuenta, no solo se detecta si cambió un dato — también se detecta si se intercambiaron dos entre sí.

```
public static int CalcularDVH(clsPacienteBE paciente)
{
    int suma = 0;
    List<string> valores = new() {
        paciente.Nombre ?? "", paciente.Apellido ?? "", paciente.DNI ?? "",
        paciente.Telefono ?? "", paciente.Email ?? "",
        paciente.FechaNacimiento.ToString("yyyyMMdd"), paciente.ObraSocial ?? ""
    };
    int posAtributo = 1;
    foreach (string valor in valores)
    {
        int posCaracter = 1;
        foreach (char c in valor) { suma += (int)c * posCaracter * posAtributo;
posCaracter++; }
        posAtributo++;
    }
    return suma % 97;
}
```

El IdPersona quedó afuera de la fórmula a propósito: al insertar un paciente, la base todavía no le asignó el Id (es IDENTITY), así que si lo incluía, el DVH calculado al guardar nunca iba a coincidir con el recalculado al leer. Lo detecté probando durante esta entrega y lo corregí ahí. El email, como está cifrado con AES, se descripta antes de calcular el DVH — si no, el dígito dependería del texto cifrado, que cambia solo en cada operación por el IV aleatorio.

El DVV (dígito verificador vertical) se guarda aparte, en una tabla DigitoVerificador (Tabla, DVV, FechaCalculo) que también forma parte del DER (G07). Es la suma de todos los DVH de los pacientes activos, módulo 97 — si alguien agrega o borra un registro por fuera del sistema, esta suma cambia y lo detecta. Usé Tabla como clave para que el mismo mecanismo sirva, en el futuro, para otras entidades sin cambiar el esquema.

Verificación de integridad y restauración

Al arrancar la app, antes del login, `VerificarIntegridad()` recorre los pacientes activos, recalcula el DVH de cada uno y lo compara contra el guardado, y después compara el DVV recalculado contra el de la tabla. Si algo no coincide, se avisa y no se deja pasar a la pantalla de login.

El recálcule (`RecalcularTodos`) es una acción manual del menú, restringida al permiso `Sistema.Mantenimiento` — nunca corre solo. Si corriera automáticamente en cada arranque, la verificación no serviría para nada: cualquier manipulación externa quedaría "corregida" antes de poder detectarla. De hecho tuve este problema al principio: tenía el recálcule enganchado en el arranque, y por eso la verificación nunca fallaba. Al recalcular, se actualiza el DVH de cada paciente y el DVV agregado, y la operación queda registrada en Bitácora.

Como Paciente no se arma a partir de varias tablas (Profesional tiene su propia tabla física completa, no comparten una tabla "Persona" en común), no tuve que resolver el problema de integridad distribuida entre tablas relacionadas para esta entidad.

Evaluación y Retrospectiva

Autoevaluación del estado del proyecto respecto de los 10 criterios definidos por la cátedra para el cierre de Ingeniería de Software, al momento de esta versión de la carpeta:

#	Criterio	Autoevaluación
1	Arquitectura diseñada	Cumplido. 4 capas (UI/BLL/DAL/BE) con dependencias unidireccionales, criterios POO aplicados (ver T01), sin frameworks de persistencia de terceros. Verificado además con pruebas unitarias automatizadas.
2	Balanceo Diagramas de Clases	Cumplido para los módulos técnicos documentados (T02, T04, T05, T06b). El diagrama de arquitectura y el de Composite reflejan exactamente las clases e interfaces del código fuente actual.
3	Balanceo DER	Cumplido. El DER de G07 fue construido en base al esquema real de la base de datos utilizada por el sistema en ejecución (incluye la tabla ControlCambios agregada en esta entrega).
4	Balanceo Casos de Uso	Cumplido para los casos de uso técnicos documentados (Login en T02, Asignar Permisos en T04). Los casos de uso de procesos de negocio completos corresponden a Trabajo de Diploma (N01/N02), fuera del alcance de esta materia.
5	Balanceo Diagramas de Secuencia	Cumplido para el diagrama de secuencia de Login (T02), que refleja la interacción real entre frmLogin, clsUsuarioBLL, clsSeguridadBLL y clsSesionActual.
6	Interfaz de Usuario	Windows Forms MDI, con habilitación/deshabilitación dinámica de menús y botones según los permisos del usuario autenticado (integración T02 + T04), y pantallas de historial/restauración de datos (T06b).
6	Correcto manejo de excepciones	Cumplido. Todos los métodos de escritura de las capas BLL usan try/catch, y tanto las operaciones normales como los errores no esperados quedan registrados en la Bitácora (clsBitacoraBLL.RegistrarError), verificado con pruebas unitarias.
7	Calidad de código y capacidad para POO	Patrones Composite (T04), Observer (T05) y Singleton (T02) aplicados correctamente y verificados con pruebas unitarias; separación de responsabilidades por capa; validaciones centralizadas en BLL.
8	Diseño de base de datos	Normalizado a 3FN, integridad referencial mediante FK, ver G07.
9	Presentación general	Este documento, siguiendo la estructura y numeración (T01-T08, G06, G07) del Plan de Entregas IS de la cátedra.